Week 3

Session 3
From Functions to the Web

Information Systems Development (SE)

Dr. Pejman Ebrahimi

Today in six parts



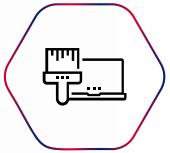
01 · Functions

One job, one name, one result



02 · Dictionaries

Records, and tables of records



03 · VS Code

A real editor and a debugger



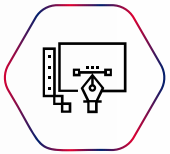
04 · Practice

Exercises, then HackerRank



05 · Flask

Python that answers a browser



06 · Build

Prompt, build, compete

Where we are

Last lecture



Conditions, loops and lists. You built a guessing game that fights back.

This week

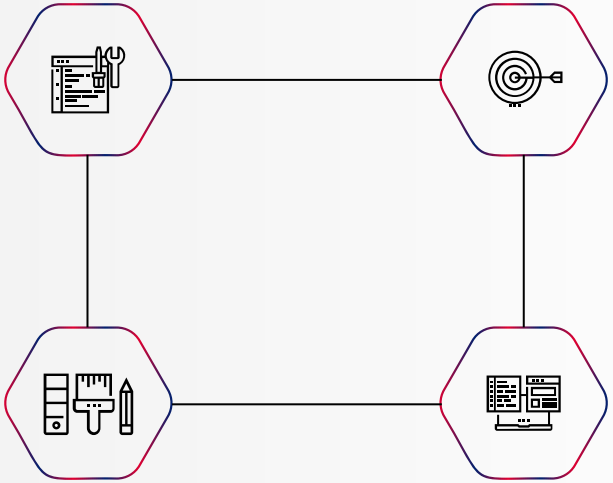


Code you can reuse, data with a shape, and your first page in a browser.

How today runs

Learn

Functions and dictionaries



Practice

Exercises, then HackerRank

Flask

A script becomes a website

Compete

Teams build and demo

© Dr. Pejman Ebrahimi · Session 3

01

Functions

Code you write once and
use everywhere

The same code, three times

```
# welcome three customers
print("Hello, Anna!")
print("Welcome back to MiniShop.")

print("Hello, Ben!")
print("Welcome back to MiniShop.")

print("Hello, Carla!")
print("Welcome back to MiniShop.")
```

Why this hurts

Three names, eight lines, one idea.

Change the welcome text and you have to change it in three places.

Miss one and nobody notices.

The fix: name it once

```
def greet(name):
    print(f"Hello, {name}!")
    print("Welcome back to MiniShop.")

greet("Anna")
greet("Ben")
greet("Carla")
```

What changed

One definition, three calls.

The welcome text now lives in exactly one place.

The name greet() says what it does, so the calls read like English.

© Dr. Pejman Ebrahimi · Session 3

The anatomy of a function

```
def total_price(price, qty):
    return price * qty

cost = total_price(899.0, 2)
print(cost)          # 1798.0
```

Four parts

def — the keyword that starts a definition.

total_price — the name you choose. Make it say what it does.

(price, qty) — the parameters it needs to do the job.

return — the value it hands back.

© Dr. Pejman Ebrahimi · Session 3

Parameters and arguments

```
def apply_tax(amount, rate):
    return amount * (1 + rate)

print(apply_tax(120, 0.2))
print(apply_tax(rate=0.2, amount=120))
```

The difference

Parameters are the names in the definition.

Arguments are the values you pass in when you call it.

Naming the arguments makes the call readable — and the order stops mattering.

© Dr. Pejman Ebrahimi · Session 3

Default values

```
def apply_tax(amount, rate=0.077):
    return amount * (1 + rate)

print(apply_tax(100))          # 107.7
print(apply_tax(100, 0.2))     # 120.0
```

The rule

A parameter with a default becomes optional.

Defaults go last, after the parameters that have none.

Choose the value you use most often.

return vs print

print shows

Puts text on the screen for a human.
Your program keeps nothing.



return gives

Hands a value back to your code, so you can store it or reuse it.



The proof

```
def shows(x):
    print(x * 2)

def gives(x):
    return x * 2

a = shows(5)        # prints 10
b = gives(5)        # prints nothing

print(a)            # None
print(b)            # 10
```

Read this twice

shows() printed 10, then handed back None.

gives() printed nothing, but handed back 10.

A function with no return always returns None.

HackerRank stubs want return, never print.

One job per function



A good name

If the name needs the word 'and', it is probably two functions.



A short body

If it does not fit on one screen, split it in two.



A clear result

Return a value, or do a thing — try not to do both at once.

02

Dictionaries

One record, many fields

A dictionary is one record

```
product = {
    "name":  "Laptop",
    "price": 899.0,
    "stock": 4,
}

print(product["name"])      # Laptop
print(len(product))         # 3
```

List or dict?

A list remembers order, so you ask for items[0].

A dictionary remembers names, so you ask for product["name"].

Use a dict when every value means something different.

© Dr. Pejman Ebrahimi · Session 3

Reading a field safely

```
product = {"name": "Laptop", "price": 899.0}

print(product["name"])              # Laptop
print(product.get("colour"))        # None
print(product.get("colour", "n/a")) # n/a

print(product["colour"])            # KeyError
```

Which one to use

Square brackets: the field must exist, or the program stops.

get(): missing is fine, and you choose the fallback.

Brackets for required fields, get() for optional ones.

Changing, adding, looping

```
product = {"name": "Laptop", "stock": 4}

product["stock"] = 3           # changes it
product["colour"] = "silver"   # adds it

for field in product:
    print(field, "=", product[field])
```

Watch out

The same line both adds and overwrites — Python does not warn you.

Looping a dictionary gives you the keys, not the values.

Use .items() when you want both at once.

A list of dictionaries is a table

```
catalogue = [
    {"name": "Laptop", "price": 899.0, "stock": 4},
    {"name": "Mouse",  "price": 24.5,  "stock": 31},
    {"name": "Cable",  "price": 9.99,  "stock": 0},
]

for row in catalogue:
    print(row["name"], row["price"])
```

The mental model

Each dictionary is a row.

Each key is a column.

This is every business system you will ever meet: orders, users, posts, invoices.

© Dr. Pejman Ebrahimi · Session 3

Asking the table a question

```
def in_stock(rows):
    available = []
    for row in rows:
        if row["stock"] > 0:
            available.append(row["name"])
    return available

print(in_stock(catalogue))
```

Everything at once

A function, a list, a dictionary, a loop and a condition — in eight lines.

This is the shape of most real reporting code.

It returns the answer and lets the caller decide what to do with it.

03

VS Code

Leaving the notebook behind

Why leave Colab

Colab was right

Nothing to install, and one cell at a time while you learned the basics.



Now it is in the way

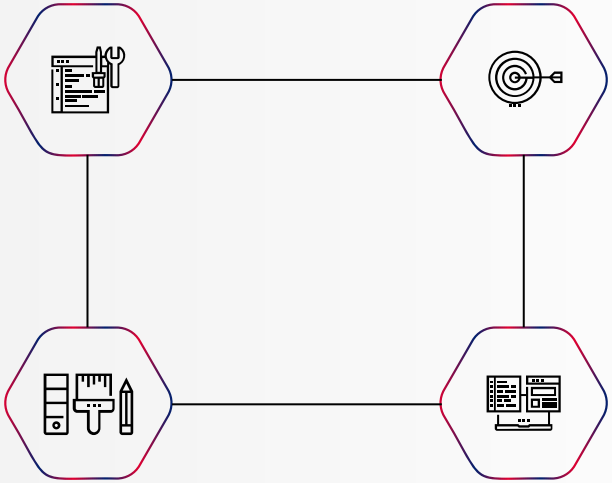
Real projects are many files, run from a terminal, with a debugger.



Set-up, in four steps

1 · Python

python.org — tick Add to PATH



2 · VS Code

code.visualstudio.com

3 · Extension

Install the Python extension

4 · Check

Run python --version

Four things to find



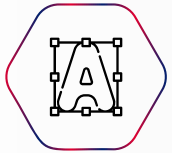
Explorer

Your files, down the left-hand side



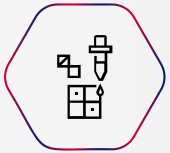
Editor

The file you are typing in



Terminal

Ctrl + ` — this is where Python runs



Run button

Top right. It just calls the terminal for you.

Running a file

```
# shop.py
def greet(name):
    return f"Hello, {name}!"

print(greet("Anna"))

# in the terminal:
#   python shop.py
#   Hello, Anna!
```

The habit

Save the file first — Ctrl + S.

Run it from the folder the file lives in.

If python is not found, the Add to PATH box was missed during install.

© Dr. Pejman Ebrahimi · Session 3

The debugger, in four moves



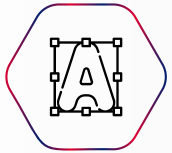
Breakpoint

Click left of a line number. A red dot appears.



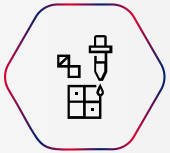
Step over

Run this line, but do not go inside it



Step into

Go inside the function being called



Variables

Watch every value change as you step

04

Practice

Your turn, at your own pace

© Dr. Pejman Ebrahimi · Session 3

What is in the exercise set



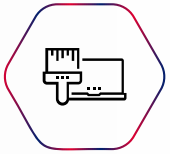
Easy

Functions that return a value



Easy

Build and read one dictionary



Medium

A table of products, filtered



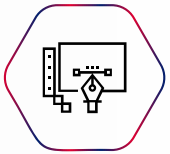
Medium

Count things with a dictionary



Hard

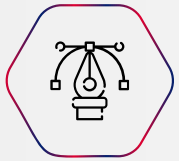
The quiz game, in three passes



Hard

Group a table by one field

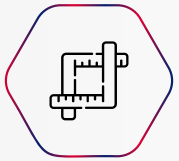
The build: a quiz game



Pass 1

Ask one question

A dictionary holding a question and its answer



Pass 2

Ask them all

A list of dictionaries, and a loop over it



Pass 3

Keep score

Three functions, and a result at the end

HackerRank today

	Stream	It teaches	Time
Write a function	A	A stub that returns, not prints	10 min
Finding the Percentage	A	Dictionaries doing real work	15 min
Sales by Match	B	Counting with a dictionary	15 min

100%

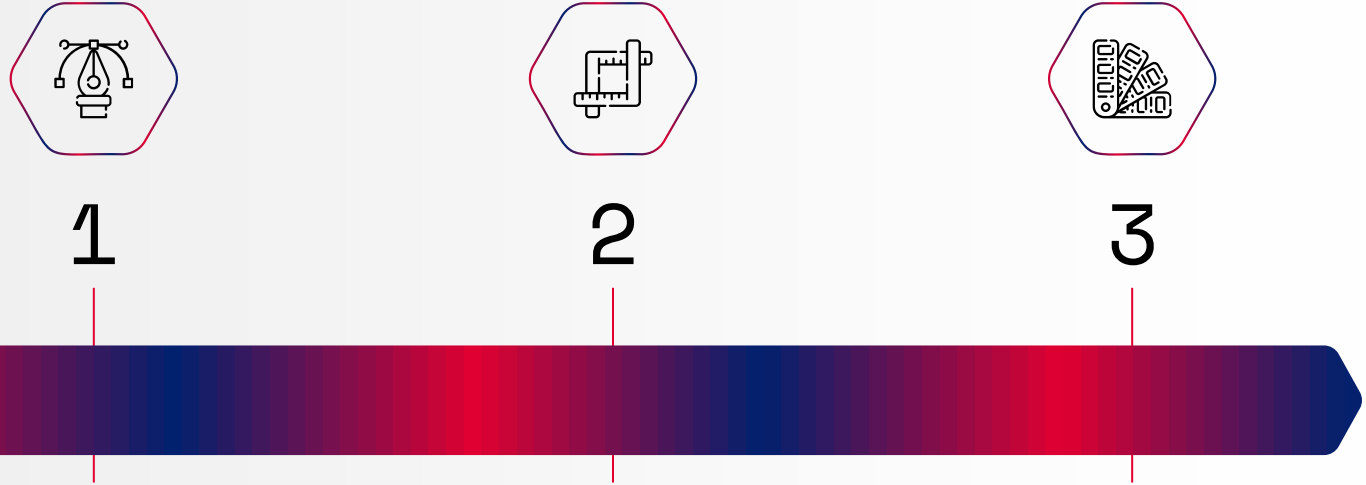
of the HackerRank marks come from submitting an attempt.
Rank is worth zero.

05

Flask

Python that answers a browser

How the web works, in one slide



1

Request

A browser asks for a URL

2

Server

Your Python decides what to send back

3

Response

HTML arrives and the browser draws it

What is Flask

A small library

It turns an ordinary Python function into something a browser can ask for.



Not a whole system

No database, no admin, no rules imposed on you. You add only what you need.



Why Flask, for this course



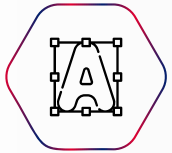
Small

Five lines and you have a website



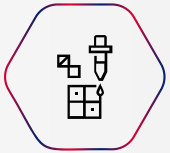
Python

No new language — just functions



Honest

You see the request and the response



Real

Used in production, not only in classrooms

Your first Flask app

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from MiniShop!"

app.run(debug=True)
```

Five things

@app.route says which URL this function answers.

Whatever the function returns is what the browser receives.

debug=True reloads on save and shows errors in the page.

Then open 127.0.0.1:5000 in your browser.

More than one page

```
@app.route("/")
def home():
    return "Welcome to MiniShop"

@app.route("/about")
def about():
    return "A tiny shop built in Session 3"

@app.route("/product/<name>")
def product(name):
    return f"You asked for {name}"
```

The pattern

One route, one function, one URL.

Angle brackets capture part of the URL and hand it to the function as an argument.

Try /product/laptop in the address bar.

HTML with holes in it

```
# app.py
@app.route("/")
def home():
    return render_template("index.html",
                           products=catalogue)

# templates/index.html
{% for p in products %}
  <li>{{ p["name"] }} — {{ p["price"] }} EUR</li>
{% endfor %}
```

Jinja

{{ }} prints a value.

{% %} runs a loop or an if.

The HTML must live in a folder called templates.

Your dictionaries from part 02 are what fills the page.

Worth knowing about Flask



Routes

A URL mapped to a function



Templates

Jinja fills HTML with your data



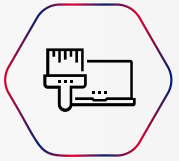
Static files

CSS and images live in /static



Forms

POST data arrives in request.form



Debug

Reloads on save, shows the error

06

Build it

Prompt, build, compete

© Dr. Pejman Ebrahimi · Session 3

Prompting is a real skill

The lazy way



"Make me a website." You get something generic that you cannot change.

The IS way



Describe the data, the pages and the rules. That is a specification — and it is your job, not the model's.

What a good app prompt contains



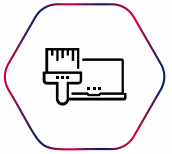
1 · Purpose

Who it is for, and what it does



2 · Data

The fields, with example rows



3 · Pages

Every URL and what it shows



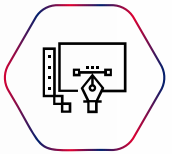
4 · Stack

Flask and Jinja, no database



5 · Limits

What it must not do



6 · Done

How you will know it works

A prompt that works

```
Build a Flask app: a reading list for a book club.

Data: a Python list of dictionaries in app.py.
Fields: title, author, year, finished (True/False).
Include six example books.

Pages:
  /           every book, newest year first
  /book/<n>   one book, by position in the list
  /unread     only the books where finished is False

Stack: Flask and Jinja only. No database, no
JavaScript, one CSS file in /static.

Done when: all three pages load and the count on
/unread matches the data.
```

Notice

It names the data before the pages.

It fixes the stack, so nothing gets invented.

It ends with a test you can actually run.

Four patterns worth stealing



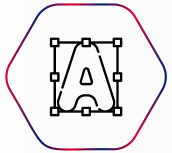
Show real rows

Paste three actual records, not a description



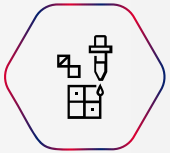
One page first

Get / working before you ask for /about



Ask why first

Have it explain the route, then write it

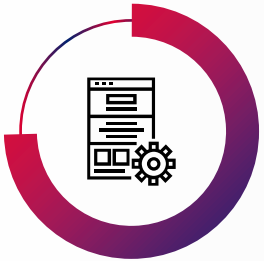


Make it fail

Ask what breaks when the list is empty

You may not ship a line you cannot explain.

The competition



20 min

Think

Pick an idea and write the prompt. No code yet.



45 min

Build

Prompt, read, fix. You do the typing.



15 min

Demo

Three minutes a team. Show it running.

The rules



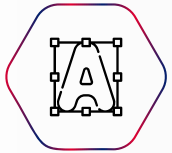
Teams of four

Mix the experienced with the new



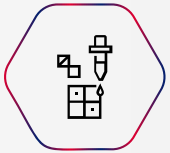
Chat AI is fine

Claude, ChatGPT or Gemini in a browser



No agents

No Claude Code, Codex or Cursor agent



You do the typing

Paste, read, understand, then run it

How it is judged



It runs

A working page beats an ambitious idea that will not load.



The idea

Something a real person would actually use. Surprise us.



You can explain it

We will point at a line and ask you what it does.

Where to run it



On your laptop

app.run(debug=True) and 127.0.0.1:5000. Enough for the demo.



Render.com

Free tier, connects to a Git repo, gives you a public URL.



PythonAnywhere

Free tier, upload files in the browser. The simplest of the three.

Homework

- Please check student notebook!

Thanks!

Questions?
pejman.ebrahimi@uni.li